

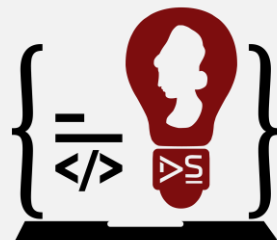


CICLO 1/ 2025 - ASIGNATURA:

LÓGICA DE PROGRAMACIÓN

UNIVERSIDAD DE EL SALVADOR - FMOcc

Departamento de Ingeniería y Arquitectura



Ingeniería en Desarrollo
de Software

```
static bool init(CURL *conn, char *url, struct http_header  
{  
    CURLcode code; struct http_header header;  
  
    conn = curl_easy_init();  
  
    if (conn == NULL)  
    {  
        fprintf(stderr, "Failed to create CURL connection\n");  
        exit(EXIT_FAILURE);  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_ERRORBUFFER,  
errorBuffer);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set error buffer [%d]\n",  
code);  
        return false;  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_URL, url);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set URL [%s]\n", errorBuffer);  
        return false;  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_FOLLOWLOCATION,  
_OK);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set redirect option [%s]\n",  
errorBuffer);  
        return false;  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_WRITEFUNCTION,  
writer);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set writer [%s]\n",  
errorBuffer);  
        return false;  
    }  
  
    code = curl_easy_setopt(conn, CURLOPT_WRITEDATA,  
&buffer);  
    if (code != CURLE_OK)  
    {  
        fprintf(stderr, "Failed to set write data [%s]\n",  
errorBuffer);  
        return false;  
    }  
  
    return true;  
}
```

Avisos y Anuncios semana 5

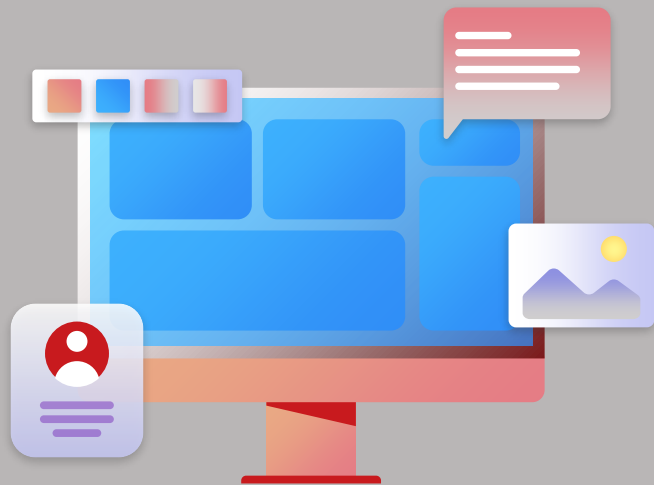


- Actividades a Realizar
 - Aplicar a la prueba Examen Corto
 - Participación en el Foro
 - Leer los contenidos de esta semana



Funciones, Arreglos, Estructuras, Memoria, Archivos, Listas

Unidad 1





Agenda.

/1.3.7

**Tamaño de
Estructuras y
Uniones.**

/1.4.1

**Gestión Dinámica
de la Memoria.**

/1.4.2

**Asignación de
Memoria para
Arreglos.**



1.3.7 Tamaño de Estructuras y Uniones.



Tamaño de Estructuras y Uniones.

Para conocer el tamaño de las estructuras y uniones utilizamos el operador: **sizeof**, este operador declarado como **sizeof()**, nos proporciona el número de bytes usados para almacenar valores para cualquier nombre de tipo de datos incluidos dentro de los paréntesis del operador. Si se usa **sizeof()** para solicitar el tamaño de una estructura o de una unión, esta función devuelve el número de bytes reservados para la estructura o la unión.

Ejemplo:

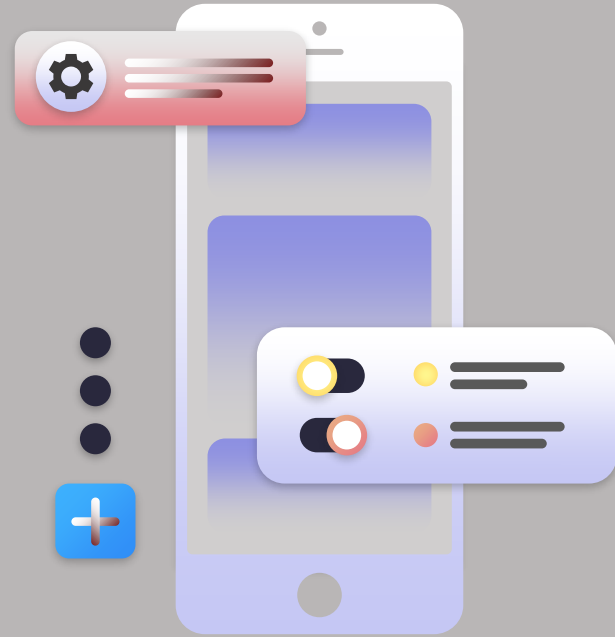
```
// declarar una unión Tipo Unión
union TiposUnion
{
    char c;
    int i;
    float f;
    double d;
};

// declarar una estructura TipoEstructura
struct TiposEstructura
{
    char c;
    int i;
    float f;
    double d;
};

cout << "sizeof(TiposEstructura): " << sizeof(TipoEstructura)<< endl;
cout << "sizeof(TiposUnion): " << sizeof(TiposUnion) << endl;
```

1.4

Memoria Dinámica.





/INTRODUCCIÓN

La memoria dinámica es un espacio de almacenamiento que se puede solicitar en tiempo de ejecución. Además de solicitar espacios de almacenamiento, también podemos liberarlos (en tiempo de ejecución) cuando dejemos de necesitarlos.

Para realizar esta administración de la memoria dinámica, C++ cuenta con dos operadores `new` y `delete`.

1.4.1 Gestión Dinámica de la Memoria.



Gestión Dinámica de la Memoria



La estructura de la memoria principal de la computadora se divide esencialmente en: memoria estática ocupada para las variables y la memoria dinámica que se gestiona con los operadores new y delete, y que se denomina también montículo o montón (heap). La memoria dinámica funciona mediante la reserva y liberación de trozos de memoria a voluntad del programador.



Gestión Dinámica de la Memoria



En C++ la memoria dinámica permanece ocupada hasta que el programador la libera explícitamente, devolviéndola a la zona de memoria dinámica libre, donde puede volver a ser utilizada.

La reserva de memoria (el tamaño) se realiza con `new` y la liberación cuando ya no se necesita se realiza con el operador `delete`, que deja libre el espacio ocupado para una posterior utilización. De igual forma `new` asigna memoria para un objeto y `delete` destruye y libera la memoria ocupada por el objeto.

Gestión Dinámica de la Memoria



Ventajas de la asignación dinámica de memoria

- El programa puede crear o destruir la asignación dinámica en cualquier momento durante la ejecución.
- Se puede determinar la cantidad de memoria necesaria en el momento en que se haga la asignación.
- Dependiendo del modelo de memoria en uso, se pueden crear variables mayores de 64 K.

Gestión Dinámica de la Memoria

- **El Operador new**

C++ proporciona otro método para obtener bloques de memoria: el operador **new**. El operador **new** asigna un bloque de memoria que es el tamaño del tipo de dato. El dato u objeto dato puede ser un **int**, un **float**, una **estructura**, un **array** o cualquier otro tipo de dato. El operador **new** devuelve un puntero, que es la dirección del bloque asignado de memoria. El puntero se utiliza para referenciar el bloque de memoria.

Sintaxis completa del operador new

```
char *ptr;
```

Diagrama de la sintaxis `char *ptr = new char [longitud];`:

- `char`: tipo puntero (se refiere a `*ptr`)
- `ptr`: nombre puntero
- `=`: operador de asignación
- `new`: palabra clave
- `char`: tipo de dato de las variables
- `[longitud]`: Número de variables tipo char (u otro tipo)

Nota: tipos de datos deben coincidir (entre `char` y `char`)

ptr es la deducción memoria
*ptr es el valor almacenado

Gestión Dinámica de la Memoria



- **El Operador delete**

El operador delete asegura un uso seguro y eficiente de la memoria; libera la memoria reservada con new. Ejemplo:

```
int * ps = new int // asigna memoria con new
... // uso de la memoria
delete ps // libera la memoria
```

Cuando se ha terminado de utilizar un bloque de memoria previamente asignado por new, se puede liberar el espacio de memoria y dejarlo disponible para otros usos, mediante el operador delete. El bloque de memoria suprimido se devuelve al espacio de almacenamiento libre, de modo que habrá más memoria disponible para asignar otros bloques de memoria.

1.4.2 Asignación de Memoria para Arreglos



Asignación de Memoria para Arreglos



- Una vez que se han reservado ubicaciones de memoria específicas para una variable, estas ubicaciones se fijan para toda la vida de esa variable, se usen o no. Una alternativa para esta asignación fija o estática de ubicaciones de almacenamiento en memoria es la asignación dinámica de memoria.
- La asignación dinámica de memoria es útil en extremo cuando se manejan listas o arrays, debido a que permite que se expandan conforme se agregan nuevos elementos y se contraiga conforme se eliminan.

Asignación de Memoria para Arreglos

```
// Asignación de memoria para un array
#include <iostream>
using namespace std;
int main()
{
    int *Dias = new int[3];
    Dias[0] = 15;
    Dias[1] = 8;
    Dias[2] = 1999;
    cout << "Las fiestas del pueblo son en Agosto "
    << Dias[0] << "/" <
    << Dias[1] << "/" <
    << Dias[2];
    delete [] Dias; // Libera memoria
    return 0;
}
```

/Recordatorios

/Realizar la actividad evaluada de la semana

/Leer los contenidos vistos en clase



/Participacion en foros

/Codificar los ejemplos expuestos y buscar otros

/GRACIAS!

/ALGUNA PREGUNTA?

